Project Proposal: High-Performance C++ Limit Order Book & Matching Engine

Project Type: Systems Programming / Quantitative Trading Infrastructure
Primary Language: C++20
Estimated Duration: 4–6 weeks, approximately 50–80 hours
Difficulty: Advanced
Target Roles: Quant Developer, Low-Latency C++ Engineer, Trading Systems Engineer, Software Engineer

---

1. Executive Summary

Build a high-performance, in-memory limit order book (LOB) and matching engine that simulates the core order-processing functionality of an electronic financial exchange.

The system will accept orders, maintain bid and ask books, match compatible orders using price-time priority, generate execution reports, and expose real-time market depth.

The project will emphasize:

- Deterministic order matching and price-time fairness.
- Efficient data structures for order insertion, cancellation, modification, and execution.
- Predictable memory allocation and low-latency processing.
- Thread-safe ingestion with a single-writer matching engine.
- Comprehensive correctness, stress, and performance testing.
- Reproducible benchmarking with latency distributions and throughput measurements.

The final deliverable will be a documented C++20 repository with a matching engine, command-line simulator, automated test suite, benchmark harness, and performance report.

2. Project Objectives

Functional Objectives

1. Support limit and market orders for buy and sell sides.
2. Implement order insertion, cancellation, and modification.
3. Match incoming orders according to price-time priority.
4. Support partial fills and multiple executions per incoming order.
5. Maintain aggregated market depth and best bid/ask quotes.
6. Generate order acknowledgments, rejections, trades, and cancellation reports.
7. Support multiple instruments using independent order books.

8. Replay deterministic order-event streams for testing.

Performance Objectives

The following are engineering targets, not measured results:

Metric	Initial target	Stretch target
Sustained throughput	100K commands/sec	1M+ commands/sec
p50 matching latency	Under 10 µs	Under 2 µs
p99 matching latency	Under 100 µs	Under 10 µs
Order lookup	Expected O(1)	Expected O(1)
Cancellation	O(1) expected with iterator-based indexing	Minimize allocation overhead
Matching correctness	100% of defined invariant tests	Differential and fuzz testing

Latency targets apply to in-process engine processing, excluding networking, disk I/O, and console output. Actual results will depend on hardware, workload, compiler flags, and measurement methodology.

3. Scope and Requirements

3.1 Order Types

Limit Order

Specifies a maximum purchase price or minimum sale price. Any unfilled quantity rests in the book unless restricted by time-in-force.

Market Order

Executes immediately against the best available opposite-side orders. Any unmatched quantity is canceled.

Time-in-Force

- GTC: Good Till Canceled; eligible unfilled limit quantity rests in the book.
- IOC: Immediate Or Cancel; unfilled quantity is canceled.
- FOK: Fill Or Kill; the entire quantity must be immediately executable or no trade occurs.

FOK orders must perform a non-mutating liquidity check before executing.

3.2 Order Modification

Support quantity and price amendments.

For the initial implementation:

- Decreasing remaining quantity at the same price preserves time priority.
- Increasing remaining quantity or changing price loses time priority.
- Price-changing amendments are implemented as cancel-and-replace.
- Modification rules must be deterministic and documented.

3.3 Order Validation

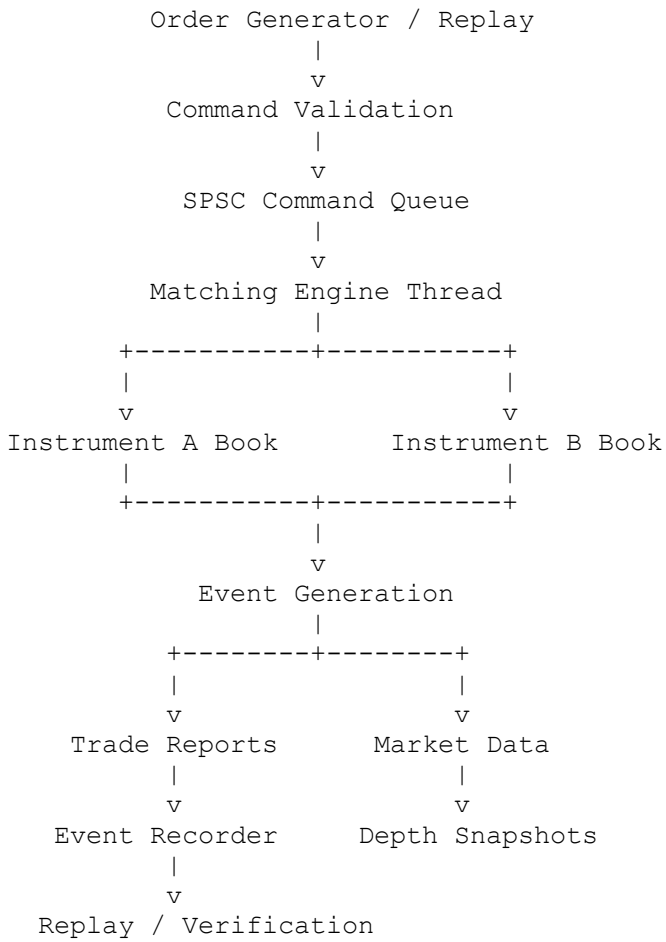
Reject commands with invalid instrument IDs, duplicate live order IDs, zero quantity, invalid tick-aligned prices, or unsupported order combinations.

Represent prices using integer ticks, not floating-point numbers.

For example, if one tick equals $0.01, a price of $125.37 is represented as integer `12537`.

4. System Architecture

Use a modular, event-driven design.



Architecture principle: One thread owns and mutates the books, eliminating locks from the matching hot path. An optional producer thread submits commands through a bounded single-producer/single-consumer (SPSC) queue.

The first milestone should use direct, single-threaded calls. Queue-based ingestion is a later optimization.

5. Core Data Structures

5.1 Order Representation

```
using OrderId = std::uint64_t;
using Price = std::int64_t;
using Quantity = std::uint64_t;
using InstrumentId = std::uint32_t;

enum class Side : std::uint8_t {
    Buy,
    Sell
};

struct Order {
    OrderId id;
    InstrumentId instrument;
    Side side;
    Price price;
    Quantity remaining_qty;
    std::uint64_t sequence;
};
```

The matching engine assigns a monotonically increasing sequence number to accepted commands, providing deterministic time priority.

5.2 Price Levels

Each price level maintains a FIFO list of resting orders.

```
struct PriceLevel {
    Price price;
    Quantity total_qty;
    std::list<Order> orders;
};
```

The first order in the list has the highest time priority at that price.

5.3 Order Book

Use ordered maps to maintain sorted price levels:

```
using BidLevels =
```

```
    std::map<Price, PriceLevel, std::greater<Price>>;

using AskLevels =
    std::map<Price, PriceLevel, std::less<Price>>;
```

The best bid and ask are accessible through `begin()`.

Maintain an auxiliary hash map:

```
std::unordered_map<OrderId, OrderHandle> order_index;
```

An `OrderHandle` identifies the side, price level, and order iterator, allowing direct cancellation without searching the entire book.

Complexity characteristics

Operation	Expected complexity
Best bid / best ask	O(1)
Find order by ID	O(1) average
Add order to existing price level	O(1) average
Create new price level	O(log P)
Cancel indexed order	O(1) average, plus O(log P) if an empty price level is removed
Match order	Depends on fills and price-level removals

Here, `P` represents the number of active price levels.

This map-and-list design prioritizes correctness and clarity. Later versions can explore flat price arrays, intrusive lists, and object pools.

6. Matching Algorithm

6.1 Price-Time Priority

For buy orders:

- Match against the lowest available ask.
- Execute while the ask price is less than or equal to the buy limit.
- Within each price level, match the oldest resting order first.

For sell orders:

- Match against the highest available bid.
- Execute while the bid price is greater than or equal to the sell limit.
- Within each price level, match the oldest resting order first.

Market orders ignore price limits but execute only against available liquidity.

6.2 Matching Pseudocode

```
process(incoming_order):

    validate(incoming_order)

    if incoming_order is FOK:
        if insufficient executable liquidity:
            reject_or_expire()
            return

    while incoming_order has remaining quantity:

        best_opposite = find_best_opposite_order()

        if no opposite order exists:
            break

        if prices do not cross:
            break

        executed_qty = min(
            incoming_order.remaining,
            best_opposite.remaining
        )

        execution_price = best_opposite.price

        generate_trade(
            incoming_order.id,
            best_opposite.id,
            execution_price,
            executed_qty
        )

        decrement both remaining quantities

        if resting order fully filled:
            remove resting order

    if incoming_order has remaining quantity:

        if incoming_order is eligible GTC limit:
            insert into its price level
        else:
            expire remaining quantity
```

Execution price follows the resting order's price.

6.3 Example

Initial book:

Bid quantity	Bid price	Ask price	Ask quantity
100	$99.50	$100.00	80
150	$99.00	$100.50	120
200	$98.50	$101.00	100

An incoming buy limit order requests 150 shares at $100.50.

Execution sequence:

1. Buy 80 shares at $100.00.
2. Buy 70 shares at $100.50.
3. Incoming order is fully filled.
4. The remaining ask quantity at $100.50 becomes 50.

The engine generates two trade events, and the order book is updated accordingly.

7. Implementation Plan

Phase 1 — Core Matching Engine

Estimated time: 8–12 hours

Implement:

- Order types and validation.
- Bid and ask price-level structures.
- Limit and market order execution.
- Partial fills and multi-level matching.
- Trade and acknowledgment events.
- Best bid/ask queries.

Acceptance criteria: All deterministic matching tests pass, including multiple resting orders at the same price.

Phase 2 — Order Lifecycle and Correctness

Estimated time: 8–12 hours

Implement:

- Order cancellation.
- Quantity and price modification.
- GTC, IOC, and FOK.
- Duplicate order ID handling.
- Invalid command rejection.

- Invariant checking.

Core invariants include:

- Resting order quantities are positive.
- Aggregate price-level quantity equals the sum of constituent orders.
- Order IDs are unique among active orders.
- Orders within each price level preserve time priority.
- The book is not crossed after processing a command.
- Executed quantity is conserved between counterparties.

Acceptance criteria: Unit tests and randomized command sequences preserve all invariants.

Phase 3 — Event Replay and Reliability

Estimated time: 6–10 hours

Implement:

- Deterministic command sequencing.
- Binary or structured-text event logs.
- Event replay.
- State snapshots.
- Checksums for comparing reconstructed book states.

Acceptance criteria: Replaying the same command sequence produces identical trade events and final order-book state.

Phase 4 — Performance Optimization

Estimated time: 10–16 hours

Establish a baseline and investigate:

- Preallocated order storage.
- Custom allocators or `std::pmr`.
- Reduced heap allocations in the matching hot path.
- Cache locality and object layout.
- Efficient price-level access.
- Avoidance of unnecessary string operations and copies.

Use profiling results to decide which optimizations to implement.

Acceptance criteria: Report before-and-after results on identical workloads, including regressions and tradeoffs.

Phase 5 — Concurrent Ingestion

Estimated time: 6–10 hours

Implement:

- A bounded SPSC ring buffer.
- A dedicated matching thread.
- An order-generating producer.
- Queue-full backpressure.
- Graceful shutdown and queue draining.

Preserve a single-writer matching architecture.

Acceptance criteria: Commands are processed in queue order without loss or duplication, and shutdown completes predictably.

Phase 6 — Documentation and Final Evaluation

Estimated time: 6–10 hours

Deliver:

- Architecture and design documentation.
- Reproducible build instructions.
- Correctness test results.
- Benchmark tables and charts.
- Optimization notes.
- Engineering tradeoff discussion.
- Resume-ready project summary.

8. Testing Strategy

Unit Testing

Use Google Test for:

- Single and multiple fills.
- Price-time priority.
- Partial fills.
- Empty books.
- Cancellation and replacement.
- IOC/FOK behavior.
- Market order expiration.
- Invalid order rejection.

- Aggregate quantity correctness.

Property-Based and Differential Testing

Generate random order sequences and compare the optimized engine against a simple, independently implemented reference matcher.

For each processed command, compare:

- Execution events.
- Best bid and ask.
- Price-level quantities.
- Active order states.
- Total traded volume.

Stress Testing

Run large deterministic workloads with:

- High cancellation rates.
- Many orders at the same price.
- Sparse and dense books.
- Frequent partial fills.
- Long-lived resting orders.
- Large order quantities.
- Bursts of incoming commands.

Use AddressSanitizer and UndefinedBehaviorSanitizer during development, plus ThreadSanitizer for concurrent components.

9. Benchmarking Methodology

Build a standalone benchmark harness that generates reproducible workloads.

Workload Profiles

Workload	Purpose
Mostly resting limit orders	Measure insertion and allocation costs
Cancellation-heavy	Measure hash lookup and removal
Aggressive market orders	Measure matching and fill generation
Mixed order flow	Approximate general exchange activity
Deep order books	Evaluate price-level scalability
Burst ingestion	Evaluate queue capacity and backpressure

Metrics

Measure:

- Commands processed per second.
- p50, p95, and p99 command-processing latency.
- Maximum observed latency.
- Memory usage.
- Allocations per command.
- Queue occupancy and dropped/rejected commands.
- Performance under different active order counts.

Benchmark Controls

- Use Release builds with documented compiler optimization flags.
- Record CPU, operating system, compiler, and hardware specifications.
- Warm up the engine before measurement.
- Use a monotonic high-resolution clock.
- Exclude input generation and console output from matching latency.
- Report repeated runs, not only the fastest run.
- Distinguish single-threaded matching throughput from end-to-end queued throughput.

Use Google Benchmark where appropriate and a custom harness for stateful order-flow simulations.

10. Repository Structure

```
limit-order-book/
├── CMakeLists.txt
├── README
```